THE NATIONAL UNIVERSITY OF LESOTHO

Assembler Lead: Design and User Guide

Role document for Milestone M2 · Setente Nthako

Team	limo-nul (lab group C)	**Release tag**	m2
Role	**Assembler lead**	**Member**	Setente Nthako (202321761)
GitHub	Setente	**Due**	Sun 11 Oct 2026, 23:59

1 Your role in M2

- Deliver a **working assembler** (`src/assembler.js`) with **unit tests** (shared with the test lead): handout B1 and the "assembler 6 marks" of the final.
- Report errors in plain language with line numbers; show machine code in binary and hex with fields.
- Produce the RV32I translation so that every program can be cross-checked in Ripes.

2 Language rules (lexical specification v1.1)

Item	Rule	Example
Comment	from `;` to end of line	`kpn R1, R2, R3 ; add`
Label	letters, digits, _ and apostrophe, starting with a letter or _, followed by a colon; may share a line with an instruction; case-sensitive; cannot be an instruction or register name	`LOOP: ts'ela: kpn R1,R1,R1`
Mnemonic	case-insensitive; the 20 LIMO names only (English/RISC-V names give a helpful message)	`KPN` = `kpn`
Registers	R0–R31 (any case); ABI aliases zero ra sp gp tp t0–t6 s0–s11 a0–a7; Sesotho forms lefela, khang1–8, nakoana1–7, bolokiloeng1–12 (no space)	`sp` = `R2`, `khang1` = `R10`
Immediates	decimal, 0x hex, 0b binary, optional sign; must fit the instruction (-2048..2047 for I/S)	`10 -3 0x1F 0b101`
Memory operand	offset(base); offset may be omitted	`4(R10)`, `(R5)`
Branch/jump target	a label, or a byte offset relative to the instruction itself (multiple of 4)	`khk R5, R7, DONE`, `tl -12`
Apostrophes	straight (') and curly (’) are the same character and part of the word; a word is never split at an apostrophe. Mnemonics contain none. In the RV32I export an apostrophe in a label becomes "_" because RISC-V assemblers reject it.	`ts'ela` → `ts_ela`

3 Algorithm

1. **Normalise** line endings and curly apostrophes.
2. **Pass 1** (every line): cut the comment at `;`; while the text starts with `NAME:` record the label with address = 4 × (instructions so far); a non-empty remainder becomes the next instruction. Errors: reserved name, duplicate label, stray colon, more than 1024 instructions.
3. **Pass 2** (every instruction): split mnemonic and operands at commas; check the operand count for the instruction kind; parse registers, immediates, memory operands; resolve a label to offset = label address − this instruction's address; check range and multiple of 4; build the 32-bit word with the format encoder.
4. **Collect**, do not stop: all errors are reported, sorted by line. The result object holds items (address, source, word, hex, binary, fields, RV32I text), words, labels, errors, hexText, rvSource.

Worked example: Program 1, address and offset arithmetic

Addr	Source	Pass 1	Pass 2 (encoding)	Hex
0	`kpnh R5, R0, 1`		I: imm 1, rs1 0, rd 5	`0x00100293`
4	`kpnh R6, R0, 0`		I: imm 0, rs1 0, rd 6	`0x00000313`
8	`kpnh R7, R0, 11`		I: imm 11, rs1 0, rd 7	`0x00B00393`
12	`LOOP: kpn R6, R6, R5`	LOOP = 12	R: rs2 5, rs1 6, rd 6	`0x00530333`
16	`kpnh R5, R5, 1`		I: imm 1, rs1 5, rd 5	`0x00128293`
20	`khk R5, R7, DONE`		B: offset = DONE 28 − 20 = +8	`0x0072D463`
24	`tl LOOP`		J: offset = LOOP 12 − 24 = −12	`0xFF5FF06F`
28	`DONE:`	DONE = 28	(label at the end, no instruction)	-

4 Error catalogue (what students see)

Mistake	Message (abridged)	Test
Misspelt instruction	Unknown instruction "kpm". Did you mean "kpn"?	C1
Typed "add"	"add" is a RISC-V (English) name. In LIMO write "kpn" instead.	C2
Too few operands	"kpn" needs 3 operands written like: kpn rd, rs1, rs2 - but you wrote 2.	C3
Register out of range	"R32" is not a register. Registers are written R0 to R31.	C4
Number instead of register	"5" is a number, but a register is needed here. Write R5 ...	C5
Immediate too big	5000 does not fit in the 12-bit immediate of kpnh. Allowed range is -2048 to 2047.	C6
Not a number	"ABC" is not a number. kpnh needs a whole number here ...	C7
Bad memory operand	"4" is not a memory operand. Write it like 4(R10) or 0(R5).	C8
Undefined label	Label "NOWHERE" is not defined. Check the spelling (labels are case-sensitive) ...	C9
Duplicate label	Label "A" is already defined on line 1.	C10
Label named like an instruction	"kpn" is an instruction or register name, so it cannot be used as a label.	C11
Offset not a multiple of 4	The jump distance 6 is not a multiple of 4 ...	C12
Target too far	The target is too far away for khl (4096 bytes; allowed -4096 to 4094) ...	C13
Double comma	There is an empty operand. Check for a double or trailing comma.	C14
More than 1024 instructions	The program is longer than 1024 instructions (instruction memory is 4 KiB).	C16

5 How to use and test the assembler

In the browser: open the site, type a program, press Assemble (Ctrl + Enter). **In Node.js (for tests):**

```
const asm = require('./src/assembler.js');
const r = asm.assemble('kpnh R5, R0, 1\nkpn R6, R5, R5');
console.log(r.ok, r.words.map(w => w.toString(16)));   // true [ '100293', '528333' ]
console.log(r.errors);                                   // [] when there are no errors
```

The assembler tests are groups A (25 encodings and round trips), B (12 syntax features) and C (16 error cases) of `tests/run_tests.js`: **53 tests, all passing**. Write a new test whenever you fix a bug.

6 Defence preparation

Likely defence question	Answer you should be able to give
Why two passes?	A branch can refer to a label defined later; the label address is only known after pass 1 has counted all instructions.
How is the branch offset computed?	Label address minus the address of the branch instruction itself (PC-relative): khk at 20, DONE at 28 gives +8.
How do you scatter the B-type immediate?	imm[12] to bit 31, imm[10:5] to bits 30:25, imm[4:1] to bits 11:8, imm[11] to bit 7; bit 0 is not stored.
What does the assembler do with an apostrophe?	It is part of the word, straight and curly are equal; it never splits a word. Mnemonics have none, labels may.
What happens with several errors?	All are collected and shown sorted by line; assembly never stops at the first.
Why reject offsets that are not multiples of 4?	Instructions are 4 bytes; an offset of 2 would jump into the middle of an instruction and LIMO has no exceptions.